

UNIVERSIDAD DE LOS ANDES
Facultad de Ingeniería y Ciencias Aplicadas
Arquitectura de Computadores 2026-20

Proyecto 3

Implementación física ASIC del Pochoco SoC

Integrantes:
Dariagny Fernández
Daniela Rangel
Javiera Sepúlveda
Profesor:
Jorge Gómez

4 de noviembre de 2026

Índice

1. Introducción **3**

2. Descripción general de la solución **3**
2.1. Del Proyecto 2 al Proyecto 3 . . . 3
2.2. Flujo completo . . . 4
2.3. Arquitectura del SoC . . . 5
2.4. Mapa de memoria . . . 6

3. Adaptación del RTL de FPGA a ASIC **6**
3.1. Cambios incorporados . . . 6
3.2. Comprobación de dependencias . . . 7

4. Estrategias de memoria **8**
4.1. Generación desde el assembler . . . 8
4.2. Las dos alternativas . . . 10
4.3. Comparación física a 100 ns . . . 10

5. Síntesis lógica **12**
5.1. Fracción del contador de ciclos . . . 13

6. Análisis temporal **14**
6.1. Restricciones . . . 14
6.2. Ruta crítica . . . 15
6.3. Optimización RTL . . . 18

7. Floorplan, placement y routing **19**
7.1. Configuración base . . . 19
7.2. Distribución de pines . . . 20
7.3. Barrido de utilización . . . 21
7.4. Costo de silicio . . . 22

8. Verificación funcional **23**
8.1. Pruebas del assembler . . . 23
8.2. Escenario del testbench . . . 23
8.3. Equivalencia RTL vs. post-síntesis . . . 24

9. Signoff y layout final **25**

10. Reproducibilidad **27**

11. Conclusiones **28**

1. Introducción

Este proyecto continúa el juego de tiempo de reacción ejecutado por el procesador Espino sobre Pochoco SoC. En el Proyecto 2 el sistema se implementó en una FPGA iCE40, donde los recursos físicos ya existen y el *bitstream* solo decide cómo conectarlos. El objetivo ahora es distinto: adaptar el mismo RTL a un flujo de implementación física ASIC con la tecnología GF180MCU, de modo que el resultado final sea un layout (GDS) con celdas estándar, pistas metálicas, vías y pines, verificado para fabricación.

El trabajo comprende cuatro bloques:

1. **Adaptación RTL:** eliminar dependencias exclusivas de la FPGA y agregar un reset externo.
2. **Exploración de memoria:** comparar una ROM de instrucciones con RAM mínima frente a una memoria unificada en flip-flops.
3. **Flujo físico:** síntesis lógica, análisis temporal, floorplan, ubicación, árbol de reloj, ruteo y *signoff* con LibreLane 3.0.16, PDK `gf180mcuD` y la biblioteca `gf180mcu_fd_sc_mcu7t5v0`.
4. **Verificación:** equivalencia funcional entre RTL y netlist post-síntesis, incluyendo el caso de botón equivocado y el reinicio de ronda.

El juego se mantiene escrito en assembly RV32E. Su assembler fue extendido para generar, además de `game.hex`, dos memorias Verilog sintetizables y un archivo de metadatos. Así, el programa puede incorporarse al ASIC sin inicializaciones propias de simulación (`$readmemh`) ni primitivas específicas de FPGA.

2. Descripción general de la solución

2.1. Del Proyecto 2 al Proyecto 3

La Figura 1 resume el flujo heredado del Proyecto 2: el assembler produce `game.hex`, que se carga en una RAM unificada inferida sobre bloques de la FPGA. En el Proyecto 3 el software no cambia, pero el programa deja de *cargarse* en una memoria preexistente y pasa a formar parte del propio hardware que se sintetiza.

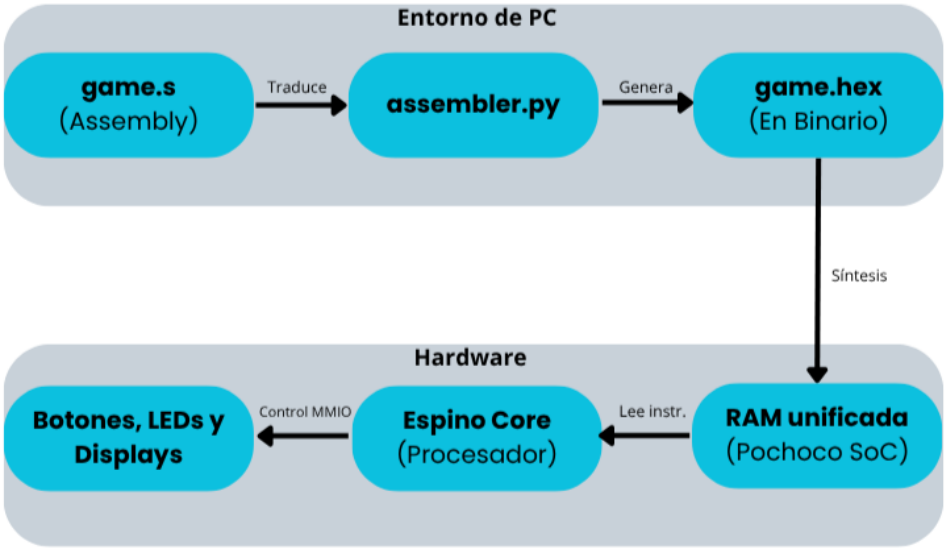


Figura 1: Flujo del Proyecto 2: del assembly a la ejecución en la FPGA.

Tabla 1: Diferencias principales entre la implementación FPGA y la ASIC.

Aspecto	Proyecto 2 (FPGA iCE40)	Proyecto 3 (ASIC GF180MCU)
Producto final	Bitstream de configuración	Layout GDS fabricable
Register file	Primitiva `SB_RAM40_4K`	Flip-flops genéricos
Programa	`game.hex` cargado en RAM	ROM o memoria FF generada en Verilog
Reset	Interno / power-on de la FPGA	Puerto externo `rst_ni` asíncrono
Restricciones	Pines de la placa	SDC, utilización, orden de pines, PDN
Verificación final	Funciona en la placa	Timing, DRC, LVS y antena

2.2. Flujo completo

La Figura 2 muestra el recorrido completo desde el programa en assembly hasta el GDS, indicando qué produce cada etapa.

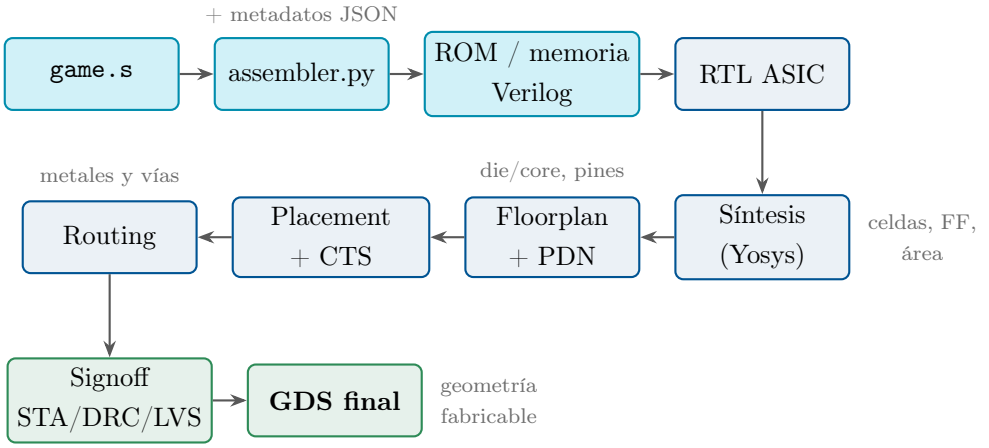


Figura 2: Flujo ASIC del Pochoco SoC: del programa en assembly a la geometría final.

2.3. Arquitectura del SoC

El módulo superior es `asic_top`. Este expone reloj, reset, botones, LED, displays y SPI como terminales lógicos (no son pads físicos) e instancia a `pochoco_soc_asic`. Dentro del SoC se integran Espino, la memoria elegida, los periféricos y SPI. El procesador utiliza un puerto de instrucciones y otro de datos. La dirección de datos se decodifica así: si el bit 31 vale 0 el acceso va a memoria; si vale 1 es MMIO, y el bit 16 separa los periféricos normales del esclavo SPI. Como la respuesta de una lectura llega un ciclo después, el SoC registra de dónde provino (`rd_is_per`, `rd_is_spi`) para multiplexar la respuesta correcta.

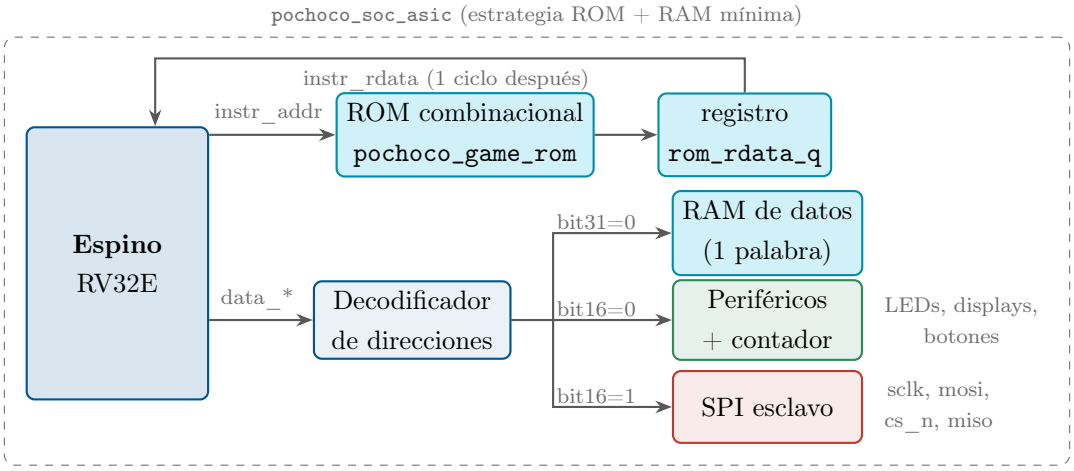


Figura 3: Diagrama de bloques del SoC en su configuración ROM + RAM mínima.

2.4. Mapa de memoria

Tabla 2: Mapa de direcciones.

Dirección	Recurso	Acceso
`0x0000_0000`	ROM de programa	L
`0x8000_0000`	Displays (2 dígitos)	E
`0x8000_0004`	LEDs	E
`0x8000_0008`	Botones	L
`0x8000_000C`	Contador de ciclos	L
`0x8001_0000`	SPI esclavo	L/E

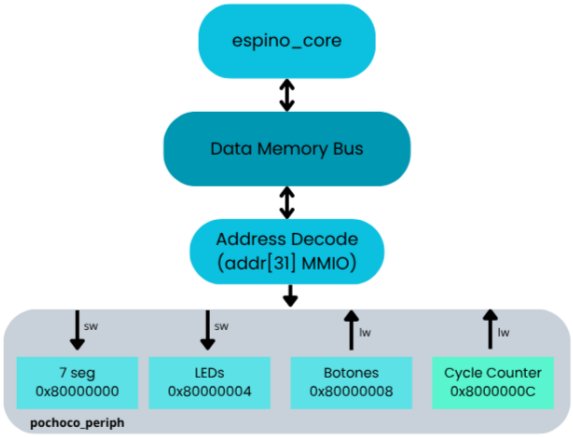


Figura 4: Decodificación MMIO de `pochoco_periph`.

3. Adaptación del RTL de FPGA a ASIC

3.1. Cambios incorporados

Tabla 3: Construcciones dependientes de FPGA y su reemplazo.

Construcción FPGA	Reemplazo ASIC	Justificación
`SB_RAM40_4K` en el register file	`espino_register_file_asic.v` con FF genéricos	La primitiva no existe en GF180. Se conserva interfaz, 2 lecturas, 1 escritura y latencia de 1 ciclo.
RAM inicializada con `$readmemh`	ROM o memoria FF generadas por el assembler	`$readmemh` no es sintetizable a celdas estándar; el contenido se expresa como lógica constante.
Reset de encendido interno (`por_cnt` en `pochoco_soc.v`)	Puerto `rst_ni` activo en bajo	El contador de 16 bits esperaba ≈2,6 ms a 25 MHz a que se estabilizaran el reloj y la block RAM de la FPGA; en un ASIC el estado inicial se controla desde fuera.
Contador dentro de periféricos	Módulo `pochoco_cycle_counter`	Permite medir su área en una síntesis jerárquica.
RAM de datos de gran tamaño	RAM de 1 palabra	`game.s` no usa la RAM de datos: todos sus accesos van a MMIO.

El register file reemplazado conserva exactamente el comportamiento del original: x0 no se almacena porque RISC-V obliga a que valga cero, la escritura se ignora si el destino es x0 y ambas lecturas se registran, entregándose un ciclo después.

Código 1: Núcleo del register file ASIC (`espino_register_file_asic.v`).

```
reg [31:0] registers [1:15];   // x1..x15; x0 no se almacena
always @(posedge clk_i or negedge rst_ni) begin
  if (!rst_ni) begin
    rdata_a_o <= 32'b0;  rdata_b_o <= 32'b0;
    for (i = 1; i < 16; i = i + 1) registers[i] <= 32'b0;
  end else begin
    if (we_i && (waddr_i != 4'd0)) registers[waddr_i] <= wdata_i;
    rdata_a_o <= (raddr_a_i == 4'd0) ? 32'b0 : registers[raddr_a_i];
    rdata_b_o <= (raddr_b_i == 4'd0) ? 32'b0 : registers[raddr_b_i];
  end
end
```

3.2. Comprobación de dependencias

La auditoría del RTL de FPGA identificó las siguientes construcciones incompatibles con un flujo de celdas estándar:

Tabla 4: Dependencias de FPGA encontradas en el RTL original y su resolución.

Ubicación en el RTL original	Problema para ASIC	Archivo ASIC
`espino_register_file.v`, líneas 22 y 33	Dos instancias de `SB_RAM40_4K` (un banco duplicado por cada puerto de lectura); además `rst_ni` no se usaba	`espino_register_file_asic.v`
`pochoco_ram.v`, líneas 35–36	Bloque `initial` con `$readmemh`: solo inicializa en simulación o en la BRAM de la FPGA	`generated/pochoco_game_rom.v`, `generated/pochoco_unified_ff_memory.v`
`pochoco_soc.v`, líneas 34–41	Reset de encendido interno pensado para la BRAM de la FPGA	`pochoco_soc_asic.v`, `asic_top.v`

En el caso del contador de *power-on reset* interno (`por_cnt` en `pochoco_soc.v`, líneas 34–41), se trata de lógica secuencial sintetizable y no de una primitiva de FPGA, por lo que la auditoría de dependencias no la rechaza. Sin embargo, mantenerlo constituye un problema crítico para una implementación ASIC ya que, a diferencia de la FPGA iCE40, donde la carga del *bitstream* fuerza un estado inicial conocido (`0`) en los registros al encender, en silicio los flip-flops energizan en un estado aleatorio o metaestable (`X`). Sin un puerto de reset externo asíncrono (`rst_ni` en `asic_top.v`), ni el propio contador `por_cnt` ni la máquina de estados de `espino_core` tendrían garantizado un punto de partida determinista al aplicar alimentación.

La validación se automatiza con `make asic-check`, que ejecuta `audit_asic_dependencies.sh`, compila ambas arquitecturas con Icarus Verilog y las sintetiza con Yosys. Este script recorre todas las fuentes ASIC y falla si encuentra `$readmemh`, instancias de `SB_RAM40_4K` o bloques `initial`; si no encuentra ninguno, informa:

```
Auditoria ASIC correcta: sin readmemh, initial ni primitivas SB_RAM40_4K.
```

La salida real de `make asic-check` para ambas arquitecturas es:

```
./scripts/check_asic_rtl.sh
Auditoria ASIC correcta: sin readmemh, initial ni primitivas SB_RAM40_4K.
Warning: Replacing memory \registers with list of registers. See rtl/asic/espino_register_file_asic
    .v:25
Warning: Replacing memory \mem with list of registers. See rtl/asic/pochoco_data_ram_ff.v:28
Warning: Replacing memory \mem with list of registers. See rtl/asic/generated/
    pochoco_unified_ff_memory.v:28
ASIC RTL valido para las arquitecturas ROM+RAM y memoria unificada.
```

Estos avisos, que Yosys repite para cada arquitectura, son informativos: indican que esas memorias se implementan con flip-flops. Las construcciones del RTL de FPGA se identificaron con la auditoría y la revisión del código de la tabla anterior. Como evidencia del register file original, se analizó `rtl/espino_core/espino_register_file.v` con Yosys 0.68, y la última línea de `fpga_error.log` es:

```
ERROR: Module ‘\SB_RAM40_4K’ referenced in module ‘\espino_regbank32’ in cell ‘\u_hi’ is not part
    of the design.
```

Este mensaje proviene de una ejecución directa de Yosys, la herramienta de síntesis del flujo, y no de una corrida de LibreLane. Muestra que la primitiva de iCE40 no existe fuera de la FPGA. Ese comando solo analiza el register file: para `$readmemh`, `initial` y el reset de encendido no se adjunta un mensaje, y por eso la tabla anterior explica por qué cada uno era un problema en una implementación ASIC.

4. Estrategias de memoria

4.1. Generación desde el assembler

El assembler resuelve las directivas `.equ`, codifica las 111 instrucciones y emite tres artefactos: `game.hex`, la ROM combinacional y la memoria unificada, además de `program_metadata.json`. Las opciones `-D NOMBRE=VALOR` permiten cambiar constantes del juego (por ejemplo `NUM_ROUNDS`) sin

editar `game.s`. Cualquier cambio exige regenerar la memoria Verilog y volver a sintetizar: editar solo `game.hex` no modifica el chip.

Código 2: Extracto de la ROM generada automáticamente.

```
module pochoco_game_rom (
  input  wire [7:0]  word_addr_i,
  output reg  [31:0] rdata_o
);
  always @* begin
    case (word_addr_i)
      8'd0:   rdata_o = 32'h80000137; // lui  x2, 0x80000  (base MMIO)
      8'd1:   rdata_o = 32'h00010193; // addi x3, x2, 0    (displays)
      8'd2:   rdata_o = 32'h00410213; // addi x4, x2, 4    (LEDs)
      8'd3:   rdata_o = 32'h00810293; // addi x5, x2, 8    (botones)
      ...                             // una linea por instruccion
      8'd110: rdata_o = 32'h00008067; // ret
      default: rdata_o = 32'h00000013; // addi x0, x0, 0 (NOP)
    endcase
  end
endmodule
```

Tabla 5: Metadatos del programa (`program_metadata.json`).

Campo	Valor
Instrucciones usadas	111
Bytes de instrucciones	444
Palabras de datos usadas por el juego	0
Palabras implementadas en la memoria unificada	128
Ocupación de la memoria unificada	$111/128 = 86{,}7\,\%$

Las cantidades de la Tabla 5 se obtienen de dos formas distintas. Las palabras de instrucciones (111, es decir, 444 bytes) las cuenta `assembler.py`: tras resolver etiquetas y directivas `.equ`, cada instrucción RV32E emitida ocupa exactamente una palabra de 32 bits. Las palabras de datos (0), en cambio, no las calcula el assembler: el campo `data_words_used_by_game` de `program_metadata.json` está fijado en 0 en `write_metadata`. Ese valor se justifica por inspección de `game.s`, donde el programa no reserva variables en RAM y todos sus `lw` y `sw` usan direcciones derivadas de la base `0x8000_0000` (cargada con `lui x2, 0x80000`), cuyo bit 31 vale 1. Según la decodificación de la Sección 2.3, esas direcciones van siempre a MMIO y nunca a la RAM de datos (bit 31 = 0).

4.2. Las dos alternativas

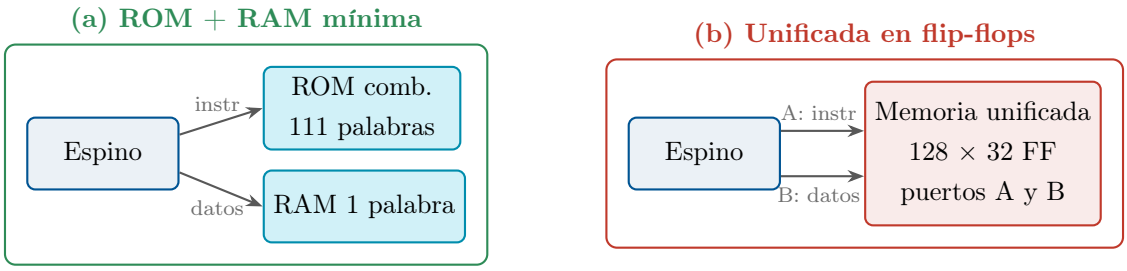


Figura 5: Las dos estrategias de memoria evaluadas.

(a) ROM de instrucciones y RAM reducida. Las 111 instrucciones se convierten en una ROM combinacional; su salida se registra en `rom_rdata_q` para conservar la latencia de un ciclo que espera Espino. La ROM no necesita flip-flops por palabra: el contenido queda expresado como lógica constante que la síntesis simplifica. Como el juego no usa memoria de datos, se mantiene una RAM de una palabra únicamente para preservar la interfaz del procesador.

(b) Memoria unificada en flip-flops. Implementa 128 palabras de 32 bits, es decir, al menos $128 \times 32 = 4096$ bits de almacenamiento en flip-flops, más los multiplexores de lectura de 128 a 1 para cada uno de los dos puertos y la lógica de escritura por byte. Es una solución general (el programa podría escribirse a sí mismo y el juego podría usar datos), pero paga ese costo aunque el juego solo ocupe 111 palabras de lectura.

4.3. Comparación física a 100 ns

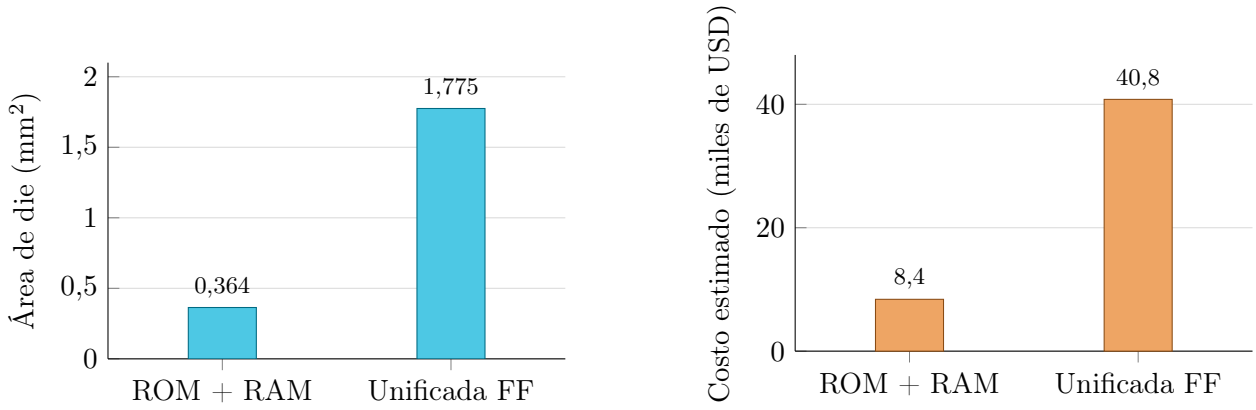


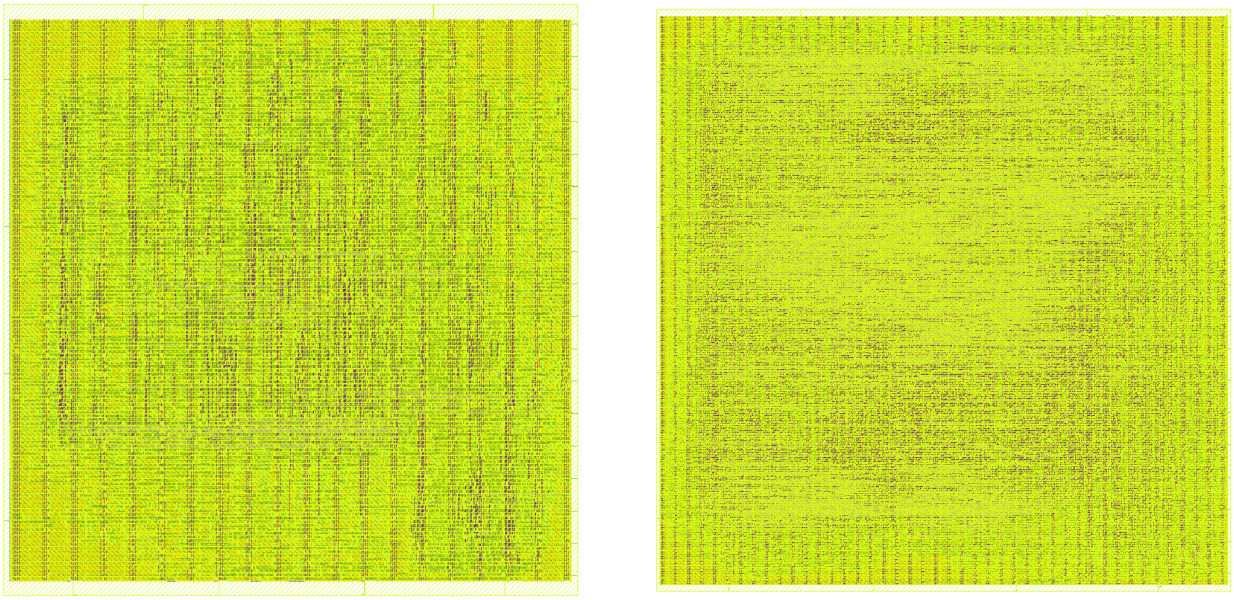
Figura 6: Área de die y costo de silicio de ambas estrategias (USD 23 000/mm^2).

Tabla 6: Comparación de las dos estrategias a 100 ns y 40 % de utilización inicial.

Métrica	**ROM + RAM mínima**	**Unificada en FF**
Área de die (μm^2)	364 210	1 775 380
Área de core (μm^2)	336 911	1 714 950
Utilización real	47,86 %	49,89 %
Bits de memoria en FF (teórico)	32 (RAM) + 32 (registro ROM)	$>$ 4096
Instancias en el layout final	18 720	93 752
Celdas tras la síntesis	4 702	22 862
Área de celdas final (μm^2)	161 244	855 607
Área lógica (μm^2)	135 101,39	686 507,09
Flip-flops totales (síntesis)	835	4 901
Área en elementos secuenciales	46,14 %	54,06 %
Frecuencia máxima reportada (menor periodo que cierra)	16,7 MHz (60 ns)	10 MHz (100 ns)
Frecuencia estimada con la holgura, típica / lenta	48,5 / 26,6 MHz	34,3 / 19,3 MHz
Holgura de setup, típica / lenta (ns)	79,378 / 62,405	70,859 / 48,252
Holgura de hold, típica / lenta (ns)	0,409 / 1,002	0,150 / 0,926
Routing DRC / Magic DRC / LVS / antena	0 / 0 / 0 / 0	0 / 0 / 0 / 0
WNS / TNS setup	0 / 0 ns	0 / 0 ns
WNS / TNS hold, peor esquina	0 / 0 ns	-0,2425 / -0,4811 ns
Setup	Cumple	Cumple
Hold	Cumple	No cumple (esquina rápida)
Costo estimado (USD)	8 376,83	40 833,74

La memoria unificada requiere $1\,775\,380/364\,210 \approx 4{,}87$ veces el área de die de la alternativa ROM. Además presenta una violación de hold en la esquina rápida (`ff_n40C_5v50`): WNS de $-0{,}2425$ ns y TNS de $-0{,}4811$ ns. En las esquinas típica y lenta sí cumple hold, con holguras de 0,150 y 0,926 ns. Una violación de hold significa que un dato cambia demasiado pronto después del flanco de reloj, típicamente por rutas muy cortas entre flip-flops vecinos combinadas con el sesgo del árbol de reloj; a diferencia del setup, no se corrige bajando la frecuencia, sino agregando retardo (buffers) en esas rutas. En las esquinas exigidas para el cierre (típica y lenta), ambas estrategias cumplen timing. Por eso la decisión se basa en el área y el costo, y la ROM con RAM reducida es la arquitectura elegida; la violación en la esquina rápida es un dato adicional.

La unificada usa 3 679 `dffrnq` y 1 222 `dffsnq`. Esta segunda celda, con set asíncrono, permite que los bits de la memoria que deben partir en 1 tomen ese valor durante el reset, de acuerdo con la inicialización que genera el assembler.



ROM + RAM mínima: 0,364 mm^2 | Unificada en flip-flops: 1,775 mm^2

Figura 7: Render automático de ambos layouts finales. Cada imagen está ajustada a su propio tamaño, por lo que no comparten escala.

5. Síntesis lógica

Para la ROM, Yosys reporta 4702 celdas y aproximadamente 135 101,39 μm^2 de área lógica. Los flip-flops dominan el área: hay 831 `dffrnq` (flip-flop D con reset asíncrono activo en bajo) y 4 `dffsnq` (con set asíncrono). En total son 835 flip-flops y 62 339,29 μm^2, es decir, el 46,14 % del área lógica con solo el 17,8 % de las celdas. En promedio, cada flip-flop ocupa 62 339,29/835 $\approx$ 74,7 μm^2.

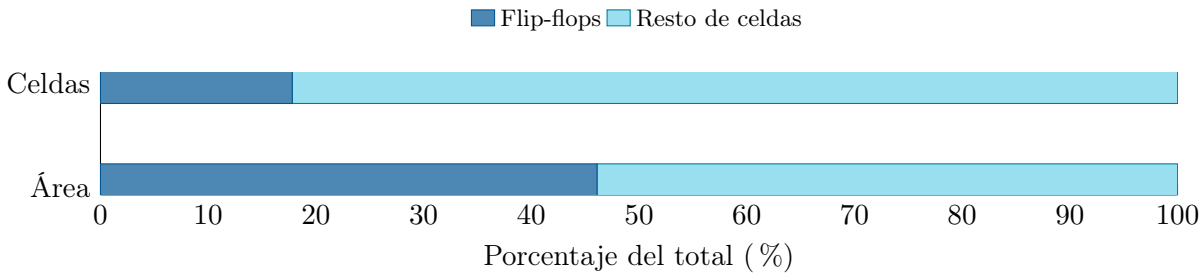


Figura 8: Peso de los flip-flops en cantidad de celdas y en área lógica (ROM, 100 ns).

Tabla 7: Cinco celdas principales de la ROM según `stat.rpt` (sin el prefijo de la biblioteca).

Celda	Cantidad	% celdas	Puesto	Área (μm^2)	% área	Puesto
`dffrnq_1`	831	17,7	1	$\approx 62\,000$	45,9	1
`nand2_1`	824	17,5	2	$\approx 9\,040$	6,7	3
`mux2_2`	435	9,3	3	$\approx 14\,300$	10,6	2
`oai21_1`	404	8,6	4	$\approx 7\,090$	5,2	5
`aoi22_1`	402	8,5	5	$\approx 7\,940$	5,9	4

Como el diseño está aplanado, la siguiente atribución se infiere del RTL y de los reportes de timing. `dffrnq_1` corresponde a los registros: 480 flip-flops son del register file (15×32) y 32 del contador. `mux2_2` se explica principalmente por la lógica de selección que mantiene o actualiza esos registros. `aoi22_1` aparece en la selección de datos, como las lecturas del register file. `oai21_1` forma cadenas en la lógica aritmética: la ruta crítica atraviesa varias, alternadas con `aoi21_1`. `nand2_1` es lógica general, como la decodificación y la ROM.

Comparación con lo esperado. Sumando los registros declarados en el RTL de la estrategia ROM se esperan cerca de 863 flip-flops, y la síntesis entrega 835. La diferencia de 28 es compatible con bits constantes o sin uso que la síntesis elimina, como los bits altos de `rdata_o` en SPI, que solo usa sus bits bajos. También era esperable que los flip-flops dominaran el área, y así ocurre, con un 46,14 %.

5.1. Fracción del contador de ciclos

La corrida final usa `SYNTH_HIERARCHY_MODE: flatten`, por lo que su reporte no separa módulos. Las áreas se obtuvieron con una síntesis jerárquica previa al aplanamiento en Yosys (`synth`, `dfflibmap` y `abc` con la biblioteca GF180), registrada en `hierarchical_area.log`. Su total, 134 787,48 μm^2, difiere en 0,23 % de los 135 101,39 μm^2 de la corrida final, porque la optimización cambia al conservar los límites entre módulos.

Tabla 8: Área por módulo en la síntesis jerárquica (estrategia ROM, sin contar submódulos).

Módulo	Área (μm^2)	% del total
`espino_register_file`	69 085,14	51,25
`espino_alu`	9 382,28	6,96
`espino_id_stage`	8 390,05	6,22
`espino_if_stage`	8 234,20	6,11
`pochoco_game_rom`	7 200,26	5,34
`pochoco_data_ram_ff`	6 664,63	4,94
`pochoco_periph`	6 014,85	4,46
`pochoco_spi_slave`	5 957,77	4,42
`pochoco_soc_asic`	5 461,66	4,05
`pochoco_cycle_counter`	3 698,91	2,74
`espino_load_store_unit`	2 500,33	1,86
`espino_decoder`	2 127,15	1,58
`espino_controller`	70,25	0,05

El módulo que aporta más área es el register file, con 51,25 %. El contador de ciclos agregado en el Proyecto 2 ocupa 3 698,91 μm^2, es decir, 2,74 %: sus 32 flip-flops más el sumador que lo incrementa.

6. Análisis temporal

6.1. Restricciones

El reloj objetivo tiene periodo de 100 ns, equivalente a $f = 1/T = 10$ MHz. Las constantes del juego están calculadas para esa frecuencia: 1 s equivale a 10^7 ciclos y 3 s a 3×10^7 ciclos.

Tabla 9: Restricciones definidas en `constraints.sdc`.

Restricción	Valor	Propósito
`create_clock`	100 ns	Objetivo temporal en `clk_i`
`set_input_delay`	0 / 2 ns	Retardo de la circuitería externa en botones y SPI
`set_output_delay`	0 / 2 ns	Tiempo que necesita el receptor externo
`set_load`	0,05	Capacitancia que manejan las salidas (en la unidad de la biblioteca)
`set_false_path`	`rst_ni`	El reset es asíncrono: no es una ruta de datos
`set_clock_uncertainty`	0,25 ns	Margen por jitter y sesgo
`set_clock_transition`	0,10 ns	Pendiente del reloj

WNS (*Worst Negative Slack*) es la peor holgura y TNS la suma de todas las holguras negativas.

Un setup negativo indica que el dato llega demasiado tarde para el siguiente flanco; un hold negativo indica que el dato cambia demasiado pronto y corrompe la captura actual. Cuando no hay violaciones, LibreLane informa WNS agregado igual a cero, por lo que la holgura real de la ruta crítica se lee en `max.rpt`.

Para que el macro conserve la temporización real del juego operando con el período objetivo $T = 100$ ns ($f = 1/T = 10$ MHz, es decir, 10 000 000 ciclos por segundo), las constantes simbólicas definidas con directivas `.equ` en `sw/game.s` se escalan según $N = t/T$ y se dividen en parte alta (20 bits para `lui`) y parte baja con signo (12 bits para `addi`):

- **Espera de tres segundos (3 s):** Se requieren

$$N_{3\text{s}} = \frac{3 \text{ s}}{100 \times 10^{-9} \text{ s}} = 30\,000\,000 \text{ ciclos} = \texttt{0x01C9C380}.$$

  Dado que los 12 bits inferiores (`0x380` = 896) son positivos, la constante se descompone como `0x01C9C` $\times\, 2^{12} + 896 = 30\,000\,000$, actualizando en `game.s` las directivas `.equ WAIT_3S_HI, 0x01C9C` y `.equ WAIT_3S_LO, 896`.

- **Tiempo en décimas de segundo (0,1 s):** Para convertir la cuenta de `pochoco_cycle_counter` (leída en `0x8000_000C`) a décimas de segundo, se requieren

$$N_{0,1\text{s}} = \frac{0{,}1 \text{ s}}{100 \times 10^{-9} \text{ s}} = 1\,000\,000 \text{ ciclos} = \texttt{0x000F4240},$$

  lo que equivale a `0x000F4` $\times\, 2^{12} + 576 = 1\,000\,000$, definiendo `.equ TENTH_HI, 0x000F4` y `.equ TENTH_LO, 576`.

- **Pausa de error de un segundo (1 s):** De forma análoga, $N_{1\text{s}} = 10\,000\,000$ ciclos = `0x00989680` = `0x00989` $\times\, 2^{12} + 1664$, fijando `ERROR_1S_HI = 0x00989` y `ERROR_1S_LO = 1664`.

Tras actualizar estos valores, el programa y el hardware de memoria se regeneran ejecutando `make -C sw clean-game && make game`, lo cual vuelve a emitir `sw/game.hex`, `rtl/asic/generated/pochoco_game_rom.v`, `pochoco_unified_ff_memory.v` y los hashes SHA-256 en `program_metadata.json`.

6.2. Ruta crítica

En la esquina típica, la ruta crítica parte del flip-flop `rom_rdata_q` (instrucción registrada), atraviesa la decodificación y la lógica combinacional del procesador y llega a otro flip-flop. El tiempo de llegada es de aproximadamente 21,214 ns y la holgura de 79,378 ns. Ambos valores suman 100,592 ns porque ese es el tiempo requerido que calcula STA: $100 + 1{,}158$ de latencia de reloj en la captura $-0{,}250$ de incertidumbre $-0{,}316$ de setup de la celda. La Figura 9 usa $T - S$ como retardo efectivo.

En la esquina lenta, la peor ruta parte de `rom_rdata_q[6]`, con llegada en 38,845 ns y holgura de 62,405 ns.

La celda de inicio es `_7720_`, cuya salida es `u_soc.gen_rom_and_ram.rom_rdata_q[5]`: el registro de instrucción de la ROM en `pochoco_soc_asic`. La celda de fin es `_8350_`, que guarda `u_soc.u_core.u_rf.registers[15][16]`: el bit 16 de x15 en el register file de Espino. La ruta parte, entonces, de la instrucción y termina en la escritura de un registro. Como Espino resuelve decode, ejecución y write-back dentro de su etapa ID/EX, corresponde a la cadena decode–ejecución–write-back. Las celdas intermedias tienen nombres genéricos por el aplanado, así que esta función se infiere de sus extremos y del RTL.

Código 3: Ruta crítica en esquina típica a 100 ns (`typical_max.rpt`). Incluye 1,104 ns de latencia de reloj antes del flip-flop.

```
Salida de celda            Retardo   Tiempo (ns)
_7720_/Q     dffrnq_1       2,281     3,385   rom_rdata_q[5], fanout 15
_3852_/ZN    clkinv_1       1,611     4,997
_5267_/ZN    nand2_1        1,985     6,986
_5387_/ZN    oai21_1        0,383     7,374
_5390_/Z     xor2_1         0,858     8,232
_5607_/ZN    oai21_1        0,207     8,439
_5610_/ZN    aoi21_1        0,478     8,918
_5613_/ZN    oai21_1        0,405     9,323
_5624_/ZN    aoi21_1        0,637     9,960
_5642_/ZN    oai21_1        0,443    10,404
_5659_/ZN    aoi211_1       1,089    11,494
_5668_/ZN    oai21_1        0,554    12,048
_5672_/ZN    nor3_1         1,003    13,052
_6447_/ZN    oai211_1       0,561    13,614
_6454_/ZN    aoi22_1        1,766    15,381
_6740_/Z     and2_1         0,496    15,881
_6754_/Z     or3_1          0,933    16,814
_6815_/ZN    oai21_1        0,375    17,190
_6816_/Z     and2_1         1,062    18,253
_6821_/ZN    aoi221_1       2,274    20,529
max_cap40/Z  buf_2          0,540    21,073
_7545_/ZN    oai21_1        0,138    21,214   llegada a _8350_/D
```

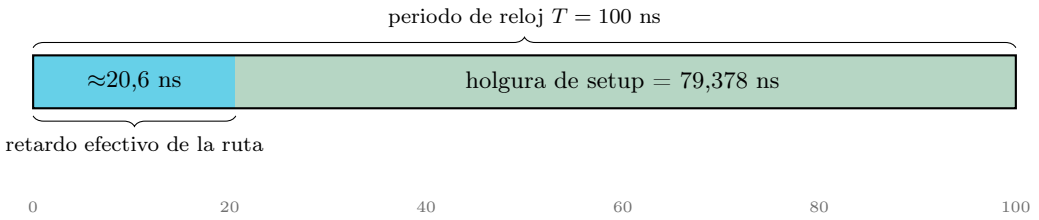


Figura 9: Presupuesto temporal de la ruta crítica de la ROM a 100 ns (esquina típica).

Frecuencia máxima estimada. Con holgura S y periodo T, el periodo mínimo aproximado es $T_{\text{mín}} \approx T - S$. Por lo tanto:

$$f_{\text{máx,típica}} \approx \frac{1}{100 - 79{,}378 \text{ ns}} \approx 48{,}5 \text{ MHz}, \qquad f_{\text{máx,lenta}} \approx \frac{1}{100 - 62{,}405 \text{ ns}} \approx 26{,}6 \text{ MHz}.$$

La frecuencia máxima reportada es **16,7 MHz**, que corresponde al menor periodo probado que cierra timing en las esquinas exigidas (60 ns). Los valores anteriores son estimaciones: el límite conservador de la esquina lenta sería 26,6 MHz, pero bajo 60 ns no hay corridas que lo confirmen.

Cada valor de `CLOCK_PERIOD` puede provocar una nueva optimización del circuito y, por tanto, modificar área, buffering y placement. En este barrido no ocurrió: las tres corridas de la Tabla 10 entregan el mismo die y la misma cantidad de instancias, y su holgura de setup baja exactamente lo que baja el periodo. Es decir, con estos objetivos el flujo llega a la misma implementación y solo cambia la restricción. La holgura de hold no depende del periodo y se mantiene igual en las tres. En los tres casos, WNS y TNS agregados de setup y hold valen 0.

Tabla 10: Barrido temporal de la estrategia ROM (utilización 40 %): holgura de la peor ruta, en ns. En todos los casos, WNS y TNS de setup y hold valen 0.

Periodo	Frecuencia	Esquina típica		Esquina lenta	
(ns)	(MHz)	Setup	Hold	Setup	Hold
100	10,0	79,378	0,409	62,405	1,002
80	12,5	59,378	0,409	42,405	1,002
60	16,7	39,378	0,409	22,405	1,002

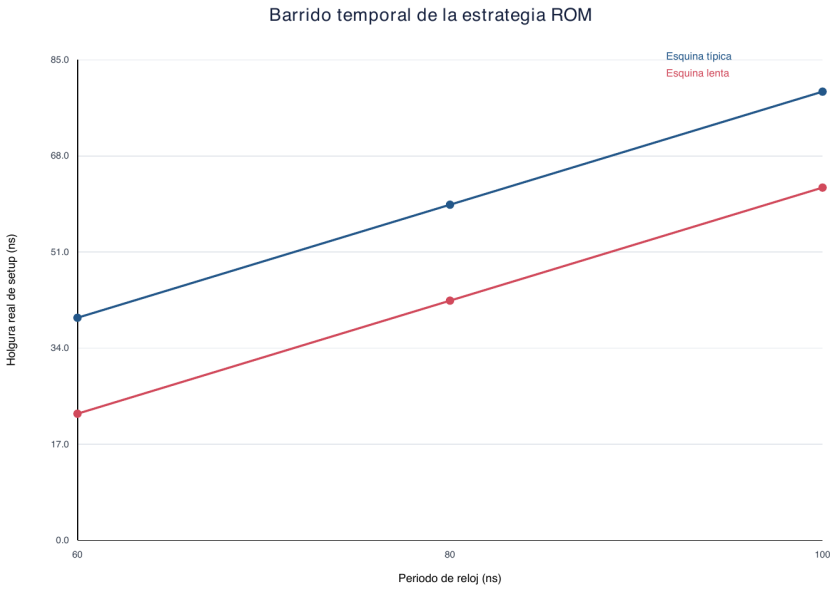


Figura 10: Holgura real de setup según el periodo de reloj, en las esquinas típica y lenta.

6.3. Optimización RTL

La ruta crítica parte de la instrucción registrada y atraviesa la decodificación. Por eso se especializó `espino_decoder.v`: la opción de síntesis `POCHOCO_GAME_MIN_ISA`, activada en `config_rom_optimized.yaml`, conserva solo las instrucciones que usa `game.s` y elimina de la lógica los demás casos. Sin esa opción, el decodificador sigue siendo el completo.

Tabla 11: Efecto de la optimización RTL (ROM, 100 ns, 40 %).

Métrica	Antes	Después
Holgura de setup, típica (ns)	79,378	78,343
Holgura de setup, lenta (ns)	62,405	60,537
Holgura de hold, típica (ns)	0,409	0,506
Holgura de hold, lenta (ns)	1,002	1,166
Frecuencia estimada, lenta (MHz)	26,6	25,3
Área lógica (μm^2)	135 101,39	129 172,15
Área de celdas final (μm^2)	161 244	154 819
Área de die (μm^2)	364 210	348 811
Costo estimado (USD)	8 376,83	8 022,65
DRC / LVS	0 / 0	0 / 0
Simulación RTL frente a netlist	PASS	PASS

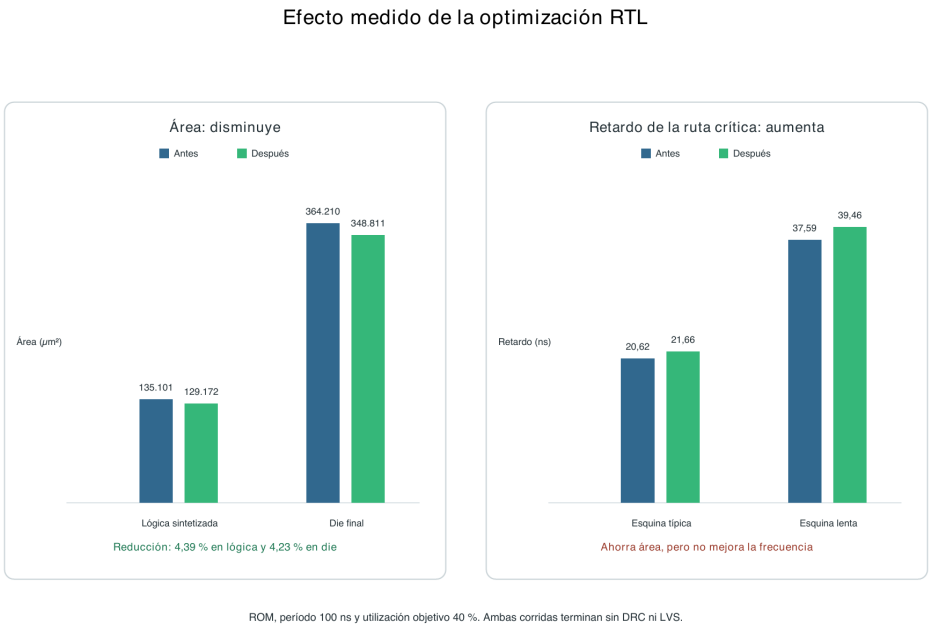


Figura 11: Efecto medido de la optimización RTL en área y retardo.

El cambio redujo el área lógica en 4,39 % y el die en 4,23 %, pero no mejoró el slack: la holgura típica bajó 1,04 ns y la lenta 1,87 ns. Al cambiar la lógica, también cambian el mapeo y el placement, y la nueva peor ruta pasa por otras celdas (`_7561_` a `_7595_`). Por lo tanto, es una optimización de área y no de frecuencia, y no se adoptó en el diseño final.

7. Floorplan, placement y routing

7.1. Configuración base

El floorplan base utiliza 40 % de utilización de core y 50 % de densidad objetivo de placement. El fanout máximo se limita a 4 para la biblioteca GF180. La síntesis reporta 135 101,39 μm^2, que sobre un core de 336 911 μm^2 equivale a un 40,1 %, coherente con el objetivo. La utilización final de 47,86 % es mayor porque las etapas posteriores agregan celdas, como los buffers y el árbol de reloj.

La red de alimentación (PDN) tiene dos niveles según `resolved.json`. Los rieles de cada fila están en `Metal1`, con 0,6 μm de ancho. Las franjas miden 2 μm, con paso de 30 μm y están definidas en `Metal4` (vertical) y `Metal5` (horizontal). Como `PDN_MULTILAYER` es `false`, LibreLane usa solo la capa inferior, es decir, las franjas verticales de `Metal4`, lo que es útil para un macro que se integrará en un diseño mayor. Las señales pueden rutearse entre `Metal2` y `Metal5` (`RT_MIN_LAYER` y `RT_MAX_`

LAYER).

7.2. Distribución de pines

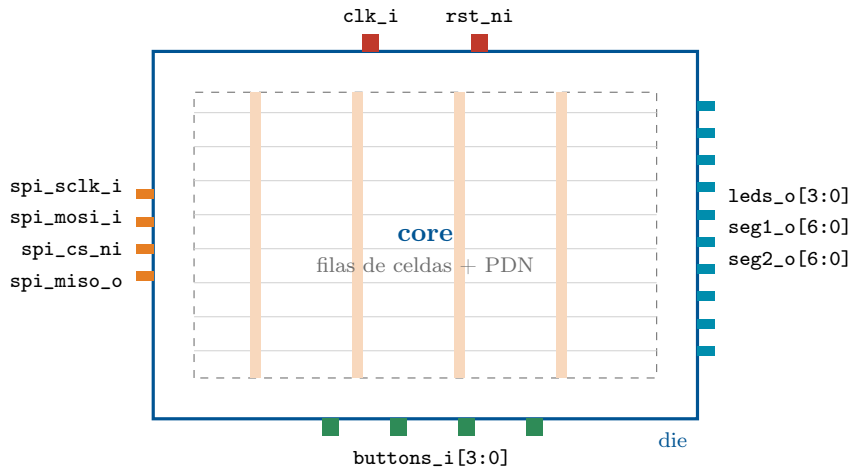


Figura 12: Distribución de pines definida en `pin_order.cfg`.

Cada grupo se ubica hacia el bloque externo con el que se conectaría: reloj y reset al norte, como señales globales de control; botones al sur, como entradas de usuario; LEDs y displays al este, agrupados por ser salidas visuales de 18 bits que conviene mantener contiguas; y SPI al oeste, separado de la interfaz de usuario. Así se evitan cruces de señales en la placa y se reducen recorridos internos largos.

Es importante distinguir estos 28 terminales ubicados en el borde del área de diseño (`pin_order.cfg`) de los *pads* de I/O que requeriría un circuito integrado completo listo para encapsular. Los pines de `asic_top` son terminales lógicos del macro implementados en capas metálicas internas del proceso GF180MCU, dimensionados para manejar cargas capacitivas internas pequeñas (`constraints.sdc` les asigna `set_load 0.05`, en la unidad de la biblioteca). En un chip completo, cada señal debería conectarse a través de un anillo de entradas/salidas (*padring*) con celdas físicas de *pad* que incorporen redes de protección contra descargas electrostáticas (ESD), *drivers* de alta corriente capaces de excitar la capacitancia de las pistas del PCB y de los LEDs y displays de 7 segmentos externos, convertidores de nivel si correspondiera, y las aberturas metálicas de *wire-bonding* hacia el encapsulado.

7.3. Barrido de utilización

Tabla 12: Barrido de floorplan de la estrategia ROM a 100 ns.

Util. objetivo	Die (μm^2)	Core (μm^2)	Util. real	Routing	DRC/LVS	Costo (USD)
30 %	480 820	449 704	36,57 %	**Cumple**	0 / 0	11 058,86
40 %	364 210	336 911	47,86 %	**Cumple**	0 / 0	8 376,83
50 %	293 911	268 903	58,73 %	**Cumple**	0 / 0	6 759,95

Las tres corridas terminan con cero errores de routing DRC, Magic DRC, LVS y antena, y cumplen setup y hold. Las holguras de setup típica y lenta son 77,803 y 59,417 ns a 30 %; 79,378 y 62,405 ns a 40 %; y 79,031 y 61,588 ns a 50 %. La menor alternativa enrutable entre las probadas es la de 50 %: reduce el die en 19,3 % y el costo en USD 1 616,88 respecto de 40 %. No se probaron utilizaciones mayores, por lo que el límite real podría estar por encima de ese punto.

Como diseño final se eligió la utilización de 40 % (corrida `final_rom_flat3`). Frente a 50 %, cuesta USD 1 616,88 más, pero conserva más espacio libre en el core y una holgura de setup levemente mayor: 79,378 frente a 79,031 ns en la esquina típica, y 62,405 frente a 61,588 ns en la lenta. La alternativa de 50 % queda documentada como la menor enrutable entre las probadas.

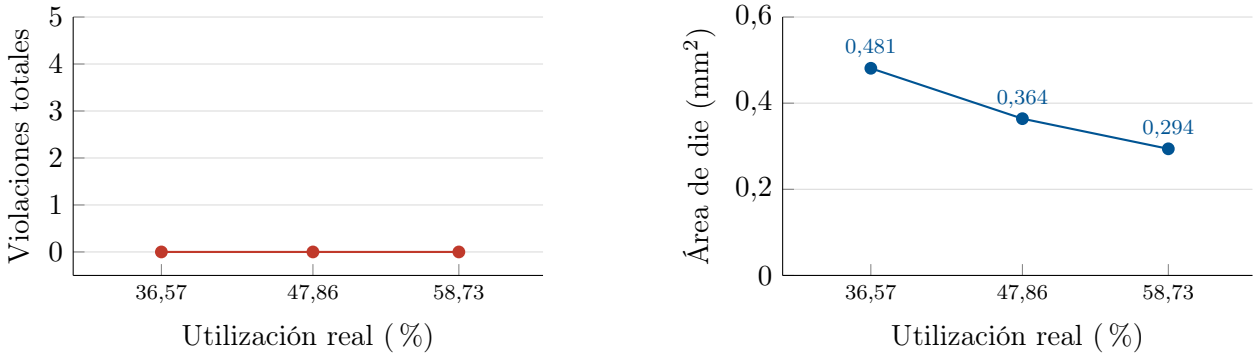


Figura 13: Utilización frente a violaciones (suma de routing DRC, Magic DRC, LVS y antena) y frente al área de die.

En los tres puntos la suma de violaciones es 0, por lo que ninguno alcanzó el límite de ruteo. El área mínima obtenida es la de 50 %, 0,294 mm^2.

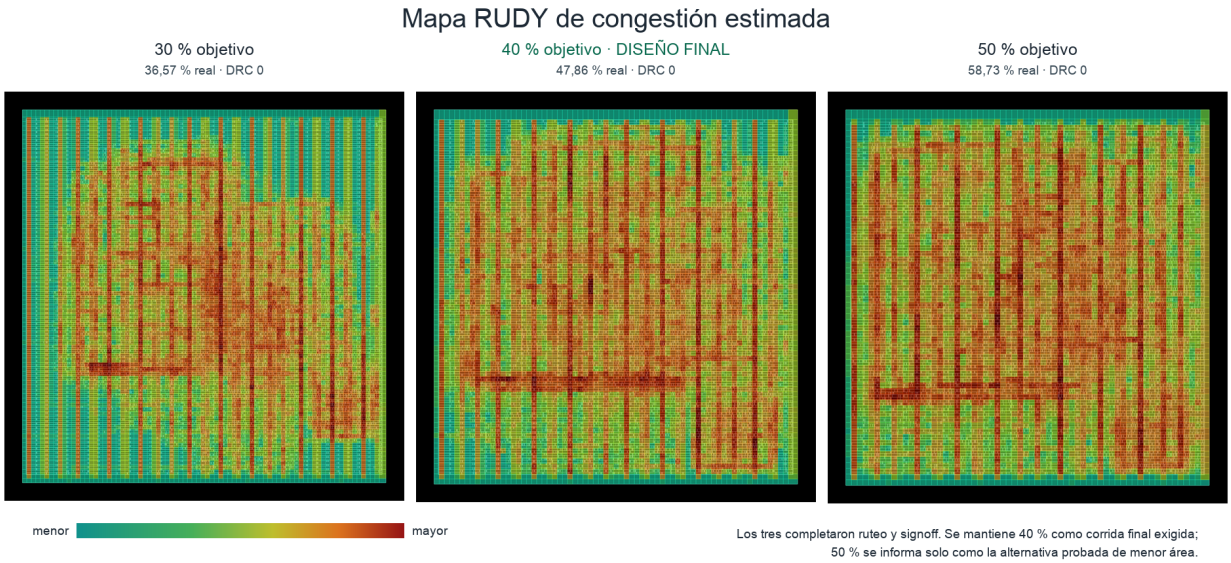


Figura 14: Mapas RUDY de congestión estimada para 30, 40 y 50 %, generados desde los ODB finales con la misma escala.

7.4. Costo de silicio

El costo se estima como $C = A_{\text{die}}[\text{mm}^2] \times 23\,000$ USD/mm^2:

$$C_{\text{ROM}} = 0{,}364210 \times 23\,000 \approx \text{USD } 8\,376{,}83$$
$$C_{\text{unificada}} = 1{,}775380 \times 23\,000 \approx \text{USD } 40\,833{,}74$$

La diferencia, de casi USD 32 457 por bloque, se debe exclusivamente a la estrategia de memoria. La comparación confirma que la decisión arquitectónica con mayor impacto sobre el área y el costo estimado fue la elección de la estrategia de memoria en `pochoco_soc_asic.v`. Reemplazar la memoria unificada en flip-flops (`u_memory`, 128×32 bits) por la ROM combinacional registrada (`u_instruction_rom` + `rom_rdata_q`) junto a una RAM mínima de 1 palabra (`u_data_ram`) redujo el área de die de 1,775 mm^2 a 0,364 mm^2, disminuyendo el costo estimado en un 79,5 % (un ahorro de USD 32 456,91). Sin embargo, suponer que el costo de fabricación escala linealmente con el área del macro (USD 23 000/mm^2) presenta limitaciones importantes frente a un proceso comercial real:

1. **Costos fijos y cuantización por bloque:** En corridas multiproyecto (*MPW/shuttle*), el espacio de oblea se contrata por bloques de tamaño mínimo fijo y no como una fracción continua en micrómetros cuadrados; en producción dedicada, los costos no recurrentes de máscaras (NRE), validación y encapsulado dominan el costo a bajo volumen.

2. **Área periférica omitida:** El área considerada corresponde únicamente al macro digital (`die_area`), sin contabilizar el anillo de *pads* de I/O (*padring*), los anillos de sello (*seal ring*) ni las calles de corte (*scribe lines*), los cuales imponen un piso de área significativo incluso si la lógica interna se reduce.

3. **Rendimiento de fabricación (*yield*):** La probabilidad de que un defecto físico inutilice un dado crece de forma no lineal con la superficie del circuito, por lo que el costo efectivo por chip funcional penaliza aún más a diseños de gran área como el de la memoria unificada.

8. Verificación funcional

8.1. Pruebas del assembler

Antes del flujo físico se verifica el assembler con sus pruebas unitarias (`make test`): ejemplos de programas (7 segmentos, parpadeo, botones y LEDs, juego completo) y validaciones de errores (registros fuera de RV32E, instrucciones y etiquetas desconocidas). Las 12 pruebas del repositorio se aprueban:

```
test_7seg (test_assembler.ExampleProgramsTest) ... ok
test_blink (test_assembler.ExampleProgramsTest) ... ok
test_buttons_leds (test_assembler.ExampleProgramsTest) ... ok
test_game (test_assembler.ExampleProgramsTest) ... ok
test_game_loads_and_stores_only_use_mmio_base_registers (test_assembler.ExampleProgramsTest) ... ok
test_equ_symbol_can_be_overridden (test_assembler.ValidationTest) ... ok
test_generated_rom_has_no_fpga_initialization (test_assembler.ValidationTest) ... ok
test_generated_unified_memory_checks_capacity (test_assembler.ValidationTest) ... ok
test_generated_unified_memory_embeds_program (test_assembler.ValidationTest) ... ok
test_rejects_registers_missing_from_rv32e (test_assembler.ValidationTest) ... ok
test_rejects_unknown_instruction (test_assembler.ValidationTest) ... ok
test_rejects_unknown_label (test_assembler.ValidationTest) ... ok
Ran 12 tests in 0.010s
OK
```

8.2. Escenario del testbench

El testbench `tb_game_asic` instancia `asic_top` con reloj de 100 ns y recorre una partida que incluye explícitamente el caso de error. Cada etapa tiene un tiempo límite para evitar simulaciones colgadas.

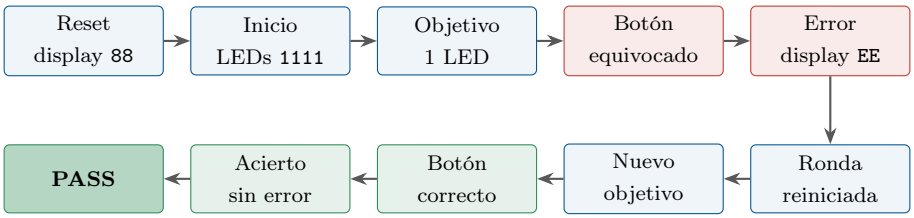


Figura 15: Secuencia de eventos verificada por el testbench.

8.3. Equivalencia RTL vs. post-síntesis

`run_functional_equivalence.sh` ejecuta el mismo escenario sobre el RTL y sobre una netlist post-síntesis, para ambas memorias. Conserva solo las líneas `EVENT` y `PASS` y las compara con `diff`: si no hay diferencias, ambos modelos tienen el mismo comportamiento observable en el escenario probado.

Esta comprobación tiene dos alcances que conviene precisar. Primero, la netlist se obtiene con el comando genérico `synth` de Yosys y se simula con `simcells.v`; no es la netlist con celdas GF180 que produce LibreLane. Segundo, para que la simulación sea ejecutable en un tiempo razonable, `generate_sim_program.sh` ensambla `game.s` con opciones `-D` que acortan las esperas a pocos ciclos (por ejemplo, `WAIT_3S_HI=0` y `WAIT_3S_LO=30`) y reducen el juego a una sola ronda (`NUM_ROUNDS=1`). La misma imagen de programa y los mismos instantes de activación de los botones del testbench (`tb_game_asic`) se aplican al RTL y a la netlist post-síntesis, sin retardos SDF anotados.

Como se verifica en el registro de eventos (`make sim-check`), ante una pulsación incorrecta el sistema detecta el fallo (`EVENT wrong_detected`), muestra `EE` en los displays y reinicia la ronda actual (`EVENT round_restarted`); posteriormente, al aplicar el botón válido en el mismo ciclo de reloj en ambos modelos, la ronda correcta finaliza exitosamente registrando exactamente el mismo número de ciclos medidos en la simulación RTL y en la post-síntesis, lo que indica que ambos modelos tienen el mismo comportamiento observable en este escenario. Esto no reemplaza una verificación de la netlist GF180 final.

Código 4: Eventos registrados en `build/sim/rom_rtl.events`.

```
EVENT startup_88 cycle=34
EVENT first_round cycle=61
EVENT first_target cycle=117 led=00000001
EVENT wrong_detected cycle=137
EVENT round_restarted cycle=171
EVENT second_target cycle=227 led=00000001
EVENT correct_detected cycle=345 seg=0111111_1001111
PASS functional_game
```

Los archivos `rom_rtl`, `rom_netlist`, `unified_rtl` y `unified_netlist` son idénticos, de modo que `diff` no reporta diferencias. La línea `correct_detected` corresponde a los dígitos 0 y 3 en los displays, es decir, 3 décimas con las constantes reducidas de la simulación.

9. Signoff y layout final

Tabla 13: Resultados de signoff de la estrategia ROM (100 ns, 40 %).

Verificación	Resultado	Qué comprueba
Routing DRC	**0**	Reglas geométricas durante el ruteo detallado
Magic DRC	**0**	Reglas geométricas de fabricación sobre el layout final
Netgen LVS	**0**	El circuito extraído del layout coincide con la netlist
Antena	**0**	No hay metales largos que acumulen carga y dañen compuertas
STA setup / hold	**0 / 0**	Sin violaciones de timing en las esquinas analizadas

En el layout, el rectángulo exterior es el die y el interior utilizable es el core; las franjas anchas forman la red VDD/VSS, las celdas estándar se alinean en filas y las líneas de colores corresponden a las capas metálicas unidas por vías.

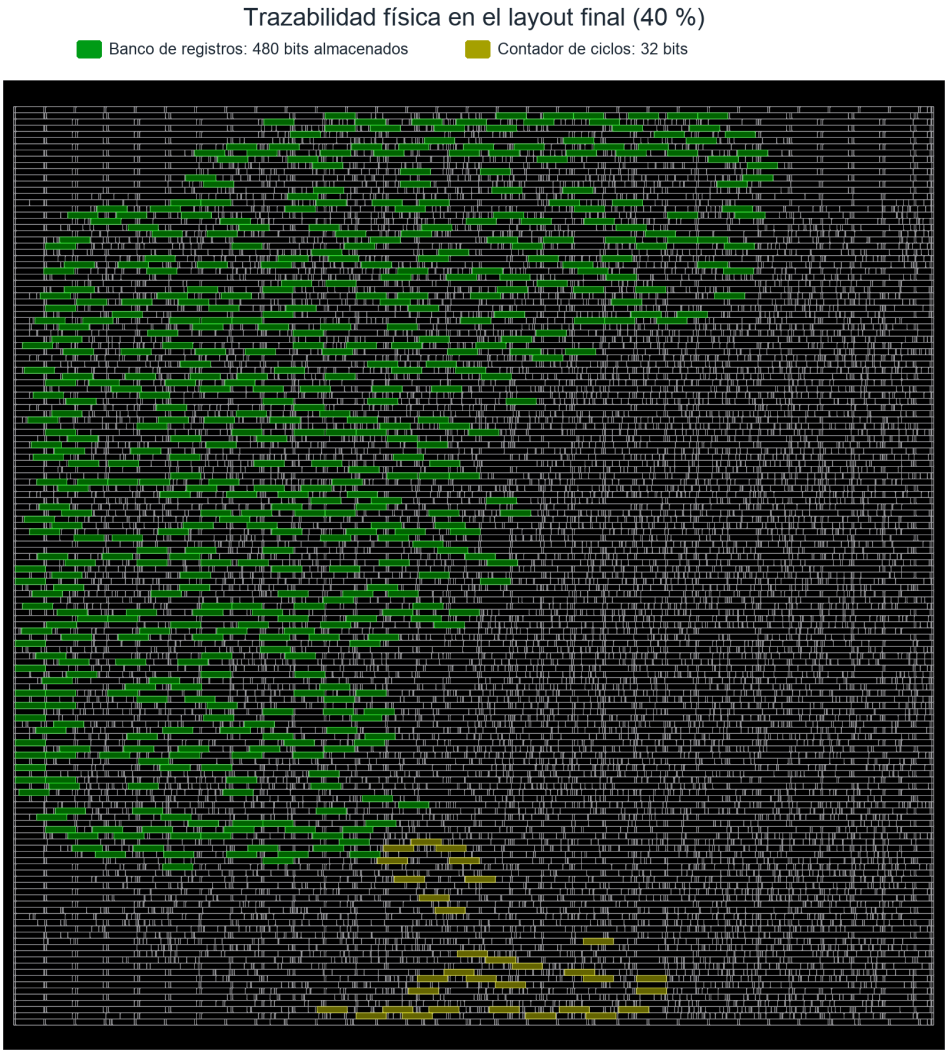


Figura 16: Layout final (40 %) en OpenROAD: en verde, los 480 bits del register file; en amarillo, los 32 bits del contador.

Como el diseño está aplanado, las celdas perdieron su nombre jerárquico, pero las redes lo conservan (por ejemplo, `u_soc.u_core.u_rf.registers[15][16]`). Por eso los flip-flops del register file y del contador se ubicaron a partir de sus redes de salida. La ALU no contiene registros y su lógica combinacional queda distribuida por el placement, así que no forma un bloque físico delimitado. Se identifica mediante la síntesis jerárquica (9 382,28 μm^2), y no se dibuja un límite físico que no existe.

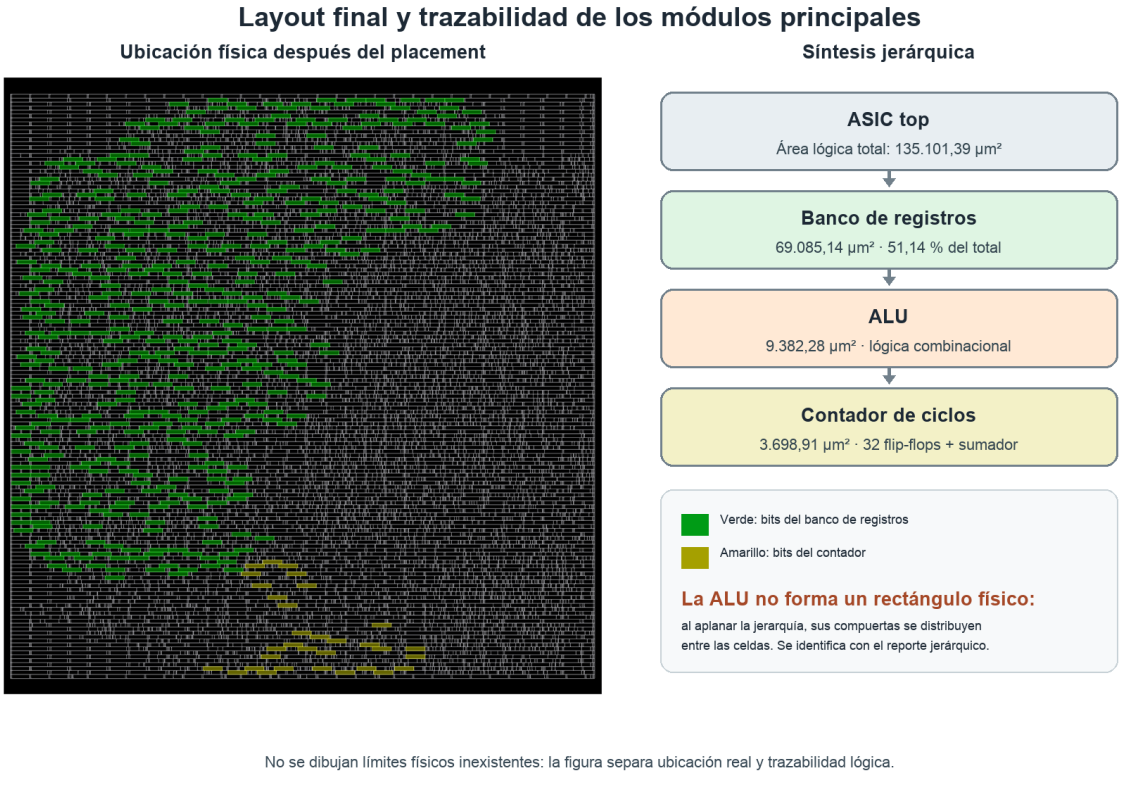


Figura 17: Ubicación física del register file y del contador, junto con la trazabilidad jerárquica que incluye a la ALU. El 51,14 % de la figura usa como total los 135 101,39 μm^2 de la corrida final; sobre el total jerárquico de la tabla de módulos es 51,25 %.

10. Reproducibilidad

El README documenta la instalación de LibreLane (mediante Nix), las versiones, la regeneración del juego, las verificaciones y las corridas. El resultado es un macro digital de celdas estándar, sin pads ni encapsulado.

Tabla 14: Versiones de herramientas utilizadas.

Herramienta	Versión
LibreLane	3.0.16
Yosys	0.62 (LibreLane); 0.68 (verificaciones y síntesis jerárquica)
OpenROAD	build 2026-02-17 (`dcf3613`)
KLayout	0.30.7
Magic	8.3.623
Netgen	1.5.316
Icarus Verilog	14.0
PDK	`gf180mcuD` (Open_PDKs `5443591`)
Biblioteca estándar	`gf180mcu_fd_sc_mcu7t5v0`

La secuencia mínima para reproducir los resultados es:

```
make game          # regenera game.hex, la ROM y la memoria unificada
make test          # pruebas del assembler
make asic-check    # auditoria, Icarus y Yosys, sin primitivas FPGA
make sim-check     # RTL vs. post-sintesis: mismos EVENT y PASS
./scripts/run_librelane.sh rom 100 40 final_rom
```

Para modificar el juego se editan sus constantes simbólicas en `game.s`, por ejemplo `NUM_ROUNDS`, `AVERAGE_DIVISOR` y `AVERAGE_STEP`, y luego se regenera el programa:

```
make -C sw clean-game
make game
make test
make sim-check
```

Los barridos completos se ejecutan con `./scripts/run_required_sweeps.sh rom` (y `unified`), y `scripts/collect_librelane_metrics.py` reúne área, utilización, WNS/TNS, signoff y costo en `evidence/librelane_metrics.csv`. Los hashes SHA-256 de `program_metadata.json` (programa: `31cd66c7...`; fuente: `3900d55a...`) permiten confirmar que el layout corresponde exactamente a la versión del programa utilizada. Desde una copia temporal limpia, siguiendo el README, se regeneró el juego y pasaron las 12 pruebas, `make asic-check` y `make sim-check` en ambas arquitecturas.

11. Conclusiones

Se adaptó el Pochoco SoC desde un RTL dependiente de la FPGA iCE40 a un flujo ASIC en GF180MCU. Las dependencias de FPGA se eliminaron: el register file pasó de dos primitivas `SB_RAM40_4K` a flip-flops genéricos con la misma latencia, se agregó un reset externo asíncrono y

el programa dejó de cargarse con `$readmemh` para generarse como hardware sintetizable desde el assembler.

La decisión de diseño más importante fue la estrategia de memoria. Los metadatos y la inspección de `game.s` muestran que el juego usa 111 palabras de instrucciones y ninguna de datos. Con esa base, y en las corridas disponibles a 100 ns, la ROM con una RAM de una palabra resultó superior a la memoria unificada en flip-flops: la unificada ocupó cerca de 4,87 veces el área de die (1,775 frente a 0,364 mm^2), elevó el costo estimado de USD 8 377 a USD 40 834 y presentó una violación de hold en la esquina rápida. La flexibilidad tiene un costo físico concreto: guardar en flip-flops un programa que nunca se reescribe consume área que el juego no aprovecha.

En la ROM, los flip-flops concentran cerca del 46 % del área lógica. El diseño cumple timing a 10 MHz con holguras de setup de 79,4 ns en la esquina típica y 62,4 ns en la lenta. También cierra a 80 y 60 ns, y su frecuencia máxima reportada es de 16,7 MHz (60 ns), el menor periodo probado. En floorplan, las utilizaciones de 30, 40 y 50 % completaron el flujo; la de 50 % es la menor alternativa probada, con 0,294 mm^2, y se eligió 40 % como diseño final, con 0,364 mm^2 y un costo estimado de USD 8 376,83. El layout de la ROM terminó con cero errores de DRC, LVS y antena.

La síntesis jerárquica muestra que el register file es el módulo dominante, con 51,25 % del área, mientras que el contador de ciclos aporta 2,74 %. La optimización del decodificador redujo el área lógica en 4,39 %, pero no mejoró el slack, por lo que se documentó como una optimización de área y no se adoptó en el diseño final, que se mantiene como ROM + RAM mínima a 100 ns y 40 %.

Pasar de FPGA a ASIC no consiste solo en cambiar la herramienta: obliga a hacer explícitas decisiones que la FPGA resolvía de antemano, como el reset, la forma física de la memoria, las restricciones temporales y la distribución de pines, y a justificarlas con mediciones de área, timing y costo.